

Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion

Boyuan Chen et al. 2023

Nikita Kiselev

Foundation models for spatial-time series
Intelligent Systems Department, MIPT

October 9, 2025

Motivation

Goal

One wants to use probabilistic sequence modeling for language processing, video prediction, decision making, etc.

Problem

- 1) **Full-sequence Diffusion**: stable generation and guidance, but requires fixed horizon and non-causal architectures.
- 2) **Teacher Forcing**: next-token models are easy to train and allow variable sequence length, but suffer from error accumulation (especially for continuous data such as video or time series).

Method

Diffusion Forcing (DF) applies independent noise to each token. The model learns to denoise them causally, combining the strengths of both worlds.

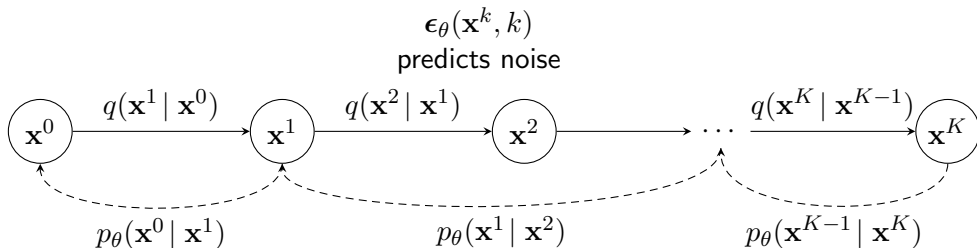
Recap: Diffusion Models

- 1) Data and schedule: $\mathbf{x}^0 \sim q(\mathbf{x})$, $\beta_k \in (0, 1)$; $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$, $k = 1, \dots, K$.
- 2) Forward (noising) process: $q(\mathbf{x}^k | \mathbf{x}^{k-1}) = \mathcal{N}(\mathbf{x}^k; \sqrt{1 - \beta_k} \mathbf{x}^{k-1}, \beta_k \mathbf{I})$.
- 3) Reverse (denoising) model: $p_\theta(\mathbf{x}^{k-1} | \mathbf{x}^k) = \mathcal{N}(\mathbf{x}^{k-1}; \boldsymbol{\mu}(\mathbf{x}^k, k), \gamma_k \mathbf{I})$.
- 4) ϵ -prediction objective:

$$\mathcal{L}(\theta) = \mathbb{E}_{k, \mathbf{x}^0, \epsilon} \left[\left\| \epsilon^k - \epsilon_\theta(\mathbf{x}^k, k) \right\|^2 \right], \quad \mathbf{x}^k = \sqrt{\bar{\alpha}_k} \mathbf{x}^0 + \sqrt{1 - \bar{\alpha}_k} \epsilon^k, \quad \epsilon^k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

Goal

Learn $p_\theta(\mathbf{x}^{k-1} | \mathbf{x}^k)$ so that sampling from noise $\mathbf{x}^K \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ recovers realistic data $\mathbf{x}^0$.



Recap: Bayesian Filtering

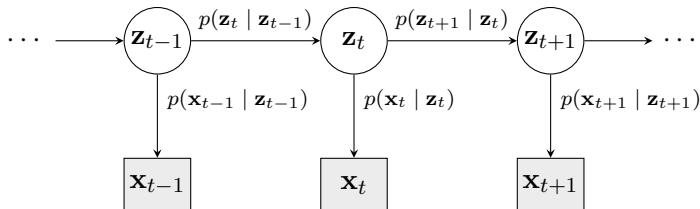
- 1) Observations: $\mathbf{x}_{1:T} = (\mathbf{x}_1, \dots, \mathbf{x}_T)$, latent states: $\mathbf{z}_{1:T} = (\mathbf{z}_1, \dots, \mathbf{z}_T)$
- 2) Transition model: $p(\mathbf{z}_t | \mathbf{z}_{t-1})$, observation model: $p(\mathbf{x}_t | \mathbf{z}_t)$
- 3) Filtering distribution: $p(\mathbf{z}_t | \mathbf{x}_{1:t})$

$$\text{Predict: } p(\mathbf{z}_t | \mathbf{x}_{1:t-1}) = \int p(\mathbf{z}_t | \mathbf{z}_{t-1}) p(\mathbf{z}_{t-1} | \mathbf{x}_{1:t-1}) d\mathbf{z}_{t-1}$$

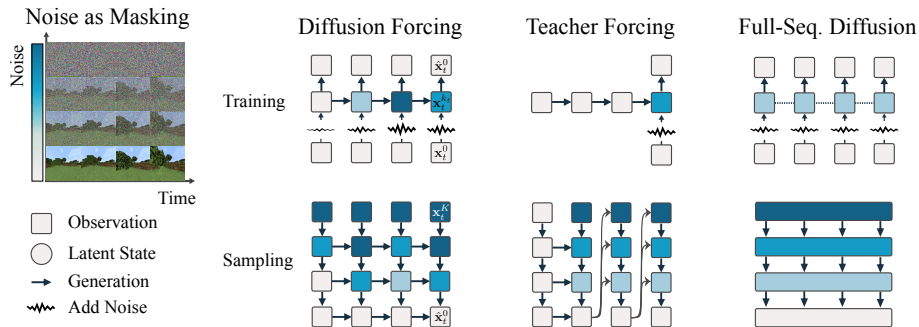
$$\text{Update: } p(\mathbf{z}_t | \mathbf{x}_{1:t}) \propto p(\mathbf{x}_t | \mathbf{z}_t) p(\mathbf{z}_t | \mathbf{x}_{1:t-1})$$

Goal

Recursively estimate $p(\mathbf{z}_t | \mathbf{x}_{1:t})$ online as new $\mathbf{x}_t$ arrive.



Method Overview



Diffusion Forcing trains causal sequence neural networks (such as an RNN or a masked transformer) to denoise flexible-length sequences where each frame of the sequence can have a different noise level. It interleaves the time axis of the sequence and the noise axis of diffusion, unifying strengths of both alternatives and enabling completely new capabilities.

Method: Diffusion Forcing (DF)

- 1) **Noising as partial masking.** View tokens as an ordered set indexed by t . Teacher forcing masks along the *time* axis (predict $\mathbf{x}_t$ from $\mathbf{x}_{1:t-1}$). Forward diffusion is *partial masking* along the *noise* axis (after K steps, $\mathbf{x}_{1:T}^K$ is $\mathcal{N}(\mathbf{0}, \mathbf{I})$).
- 2) **Notation.** Sequence $\mathbf{x}_{1:T}$; per-token noise level $k_t \in \{0, \dots, K\}$; noisy token $\mathbf{x}_t^{k_t}$ with $\alpha_k = 1 - \beta_k$, $\bar{\alpha}_k = \prod_{j=1}^k \alpha_j$.
- 3) **Causal DF (CDF).** Train a causal model so that $\mathbf{x}_t^{k_t}$ depends only on *past* noisy tokens. Maintain latent state $\mathbf{z}_t$ with dynamics $\mathbf{z}_t \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$, e.g. RNN.
- 4) **Training objective (noise prediction).**

$$\mathcal{L}(\theta) = \mathbb{E}_{\substack{k_t, \mathbf{x}_t, \epsilon_t \\ \mathbf{z}_t \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)}} \left[\sum_{t=1}^T \left\| \epsilon_t - \epsilon_\theta(\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t) \right\|^2 \right],$$

where $\mathbf{x}_t^{k_t} = \sqrt{\bar{\alpha}_{k_t}} \mathbf{x}_t^0 + \sqrt{1 - \bar{\alpha}_{k_t}} \epsilon_t$, $\epsilon_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

- 5) **Key properties.** Optimizes a (reweighted) ELBO over the likelihoods of *all* subsequences/noise schedules at once. $k_t \equiv 0 \rightarrow$ posterior update in Bayesian Filtering; $k_t \equiv K \rightarrow$ modeling prior distribution $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1})$.

Algorithm 1 CDF Training (noise prediction)

```
1: repeat
2:   Sample a trajectory  $(\mathbf{x}_1, \dots, \mathbf{x}_T)$ 
3:   for  $t = 1$  to  $T$  do
4:     Sample  $k_t \in \{0, \dots, K\}$  independently
5:      $\mathbf{x}_t^{k_t} \leftarrow \text{ForwardDiffuse}(\mathbf{x}_t, k_t)$ 
6:      $\epsilon_t \leftarrow \frac{\mathbf{x}_t^{k_t} - \sqrt{\bar{\alpha}_{k_t}} \mathbf{x}_t}{\sqrt{1 - \bar{\alpha}_{k_t}}}$ 
7:      $\mathbf{z}_t \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$ 
8:      $\hat{\epsilon}_t \leftarrow \epsilon_\theta(\mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$ 
9:   end for
10:   $\mathcal{L} \leftarrow \text{MSE}([\hat{\epsilon}_t]_1^T, [\epsilon_t]_1^T)$ 
11:  Update  $\theta$  by backprop on  $\mathcal{L}$ 
12: until converged
```

- 1) The dynamics model $p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t^{k_t}, k_t)$ and the observation model $p_\theta(\mathbf{x}_t^0 \mid \mathbf{z}_t)$ together form an RNN unit.
- 2) Latents $\mathbf{z}_t$ are RNN's hidden states.
- 3) It uses conditioning variable $\mathbf{z}_{t-1}$ and a noisy token $\mathbf{x}_t^{k_t}$ as input to predict the noise-free $\mathbf{x}_t = \mathbf{x}_t^0$.

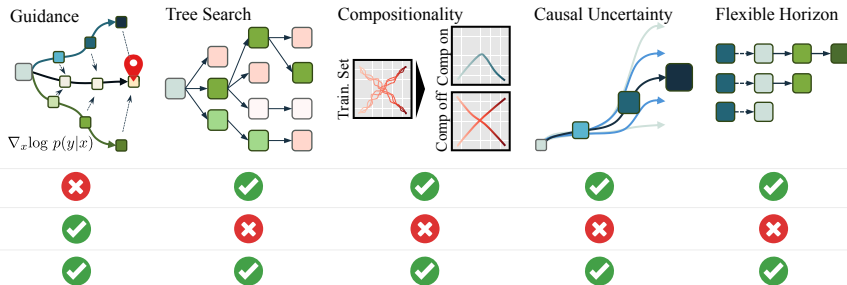
Sampling

- 1) Define a noise schedule on a 2D $M \times T$ grid $\mathcal{K} \in [K]^{M \times T}$: columns correspond to time step t and rows indexed by m
determine noise-level, $\mathcal{K}_{0,t} \equiv 0$.
- 2) Iterate down the grid row-by-row, denoising left-to-right across columns.
- 3) Framework allows to use classifier guidance: given a classifier $c(y|\mathbf{x}^k)$, one modifies the Langevin sampling gradient $\epsilon_\theta(\mathbf{x}^k, k)$ to be $\epsilon_\theta(\mathbf{x}^k, k) - \sqrt{1 - \bar{\alpha}_k} \nabla_{\mathbf{x}^k} \log c(y|\mathbf{x}^k)$.

Algorithm 2 CDF Sampling with Guidance

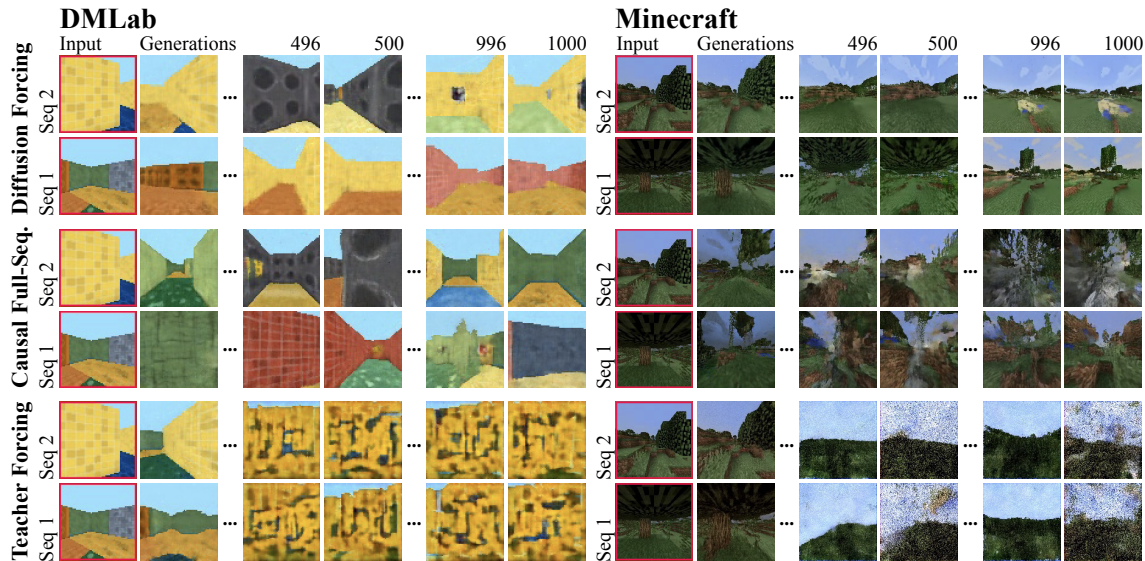
```
1: Input: model  $\theta$ , grid  $\mathcal{K} \in [K]^{M \times T}$ , init latent  $\mathbf{z}_0$ , cost  $c(\cdot)$ 
2: Initialize  $\mathbf{x}_{1:T} \sim \mathcal{N}(\mathbf{0}, \sigma_K^2 \mathbf{I})$ 
3: for  $m = M - 1$  to 0 do
4:   for  $t = 1$  to  $T$  do
5:      $\mathbf{z}_t^{\text{new}} \sim p_\theta(\mathbf{z}_t \mid \mathbf{z}_{t-1}, \mathbf{x}_t, \mathcal{K}_{m+1,t})$ 
6:      $k \leftarrow \mathcal{K}_{m,t}$ ,  $\mathbf{w} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 
7:      $\mathbf{x}_t^{\text{new}} \leftarrow \frac{1}{\sqrt{\alpha_k}} \left( \mathbf{x}_t - \frac{1 - \alpha_k}{\sqrt{1 - \bar{\alpha}_k}} \epsilon_\theta(\mathbf{z}_t^{\text{new}}, \mathbf{x}_t, k) \right) + \sigma_k \mathbf{w}$ 
8:      $\mathbf{z}_t \leftarrow \mathbf{z}_t^{\text{new}}$ 
9:   end for
10:   $\mathbf{x}_{1:H} \leftarrow \text{AddGuidance}(\mathbf{x}_{1:H}^{\text{new}}, \nabla_{\mathbf{x}} \log c(\mathbf{x}_{1:H}^{\text{new}}))$ 
11: end for
12: return  $\mathbf{x}_{1:T}$ 
```

Applications

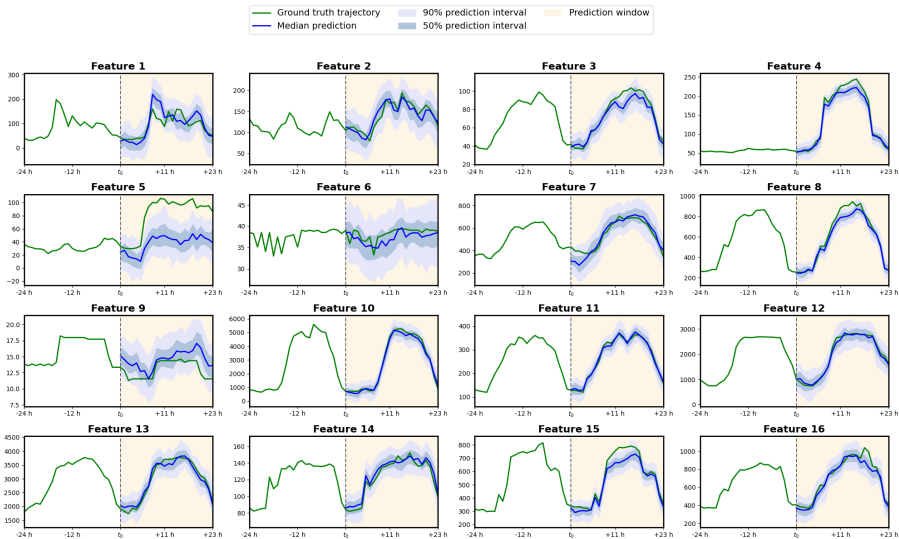


- 1) **Time series:** stable long-horizon forecasting.
- 2) **Video:** temporally consistent rollouts beyond the training horizon.
- 3) **Planning:** guidance and Monte Carlo Guidance (MCG).
- 4) **Robotics:** imitation learning with memory and robustness to noise.

Video Generation



Multivariate Probabilistic Time Series Forecasting



Conclusion & References

- 1) Diffusion Forcing unifies next-token prediction and full-sequence diffusion.
- 2) Provides stable generation and flexible horizons.
- 3) Enables guidance and decision-making in sequential tasks.
- 4) Promising for time series, video modeling, and reinforcement learning.



Chen, B., Marti Monso, D., Du, Y., Simchowitz, M., Tedrake, R., Sitzmann, V.
Diffusion Forcing: Next-token Prediction Meets Full-Sequence Diffusion.
NeurIPS 2024. <https://doi.org/10.48550/arXiv.2407.01392>



Wu, T., Fan, Z., Liu, X., Zheng, H. T., Gong, Y., Jiao, J., et al.
AR-Diffusion: Auto-Regressive Diffusion Model for Text Generation.
NeurIPS 2023. <https://doi.org/10.48550/arXiv.2305.09515>



Ho, J., Jain, A., Abbeel, P. *Denoising Diffusion Probabilistic Models.*
NeurIPS 2020. <https://doi.org/10.48550/arXiv.2006.11239>



Rasul, K., Seward, C., Schuster, I., Vollgraf, R.
Autoregressive Denoising Diffusion Models for Multivariate Time Series Forecasting.
ICML 2021. <https://doi.org/10.48550/arXiv.2101.12072>